

Variability and Software Product Line Engineering

Prof Camille SALINESI
Camilo CORREA RESTREPO
© Prof Raul MAZO PENA

UNIVERSITÉ PARIS 1
PANTHÉON SORBONNE

Centre de Recherche en Informatique

1

1

Prof. Camille Salinesi

- SW Engineering with focus on IS
- RE expert
- 150+ publications
- H-index 22
- Head of CRI
- Former VP of IREB
- VP @ University Paris 1 Panthéon - Sorbonne

UNIVERSITÉ PARIS 1
PANTHÉON SORBONNE

Centre de Recherche en Informatique

Camille.Salinesi@univ-paris1.fr

2

2

Research contributions @ P1 Panthéon- Sorbonne

- ***VARIAMOS***
- CREWS-*L'Ecritoire*
- MAP goal modelling
- MENTOR
- O*
- REMORA



- Early 90's
  - OO methods
  - Event-driven design
- Late 90's
  - Goal oriented RE
  - Use Cases and scenarios
- Early 2000:
  - Enterprise Architecture
  - ***Software Product Lines***
- Late 2000
  - Security RE
  - Sustainable software Design
- Early 10's
  - Innovation
  - Self-adaptive Systems

3

3

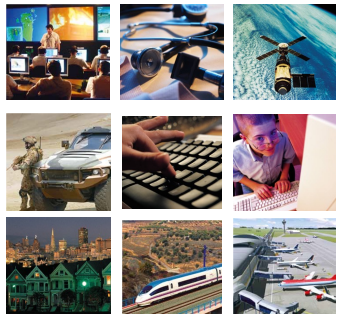
Industry collaborations

THALES
DASSAULT SYSTEMES
RENAULT
Stago
EDF
specif
IREB International Requirements Engineering Board
Powered by GNU
orange
KTM ADVANCE
france telecom
Alcatel·Lucent
Nomia
REXEL

4

4

Domains of application



5

Goal and organization of the course

Help you integrate
Product Line Engineering (PLE):

✓in your development practice &

✓in your research agenda

Part 1: Introduction to variability and PLE

Part 2: PLE processes and how to implement them

Part3: Application Engineering

6

6



7

Today

1. Introduction
2. Craft
3. Engineering
4. Reuse and product lines
5. Benefits of product line engineering



8


Let's play!
Find a way to describe both AT THE SAME TIME !
9

9


Let's play!
Find a way to describe both AT THE SAME TIME !
10
hat size
nose
moustache
button
gloves
stripes
shadow

10

Commonality

Persons and things have *common* characteristics or *features*

Examples
- They are British guards
- They have moustache
- They have a jacket, hat, pants, belts and shoes
- They have 2 eyes each one
- They have the right leg raised

11

11

Variability

Besides, they are distinguished by their varying *features*

Examples:
- Hat size
- Moustache type
- Stripes color

12

12

Dependencies

Features can be related, or depend on each other

- Big hat *projects* big shadow

13

13

Summary



Commonality **Variability** **Dependencies**

14

14

Let's play another game!

Task

Set up a sandwich store



Requirements

1. Offer a variety of sandwiches
2. Minimize ingredients waste

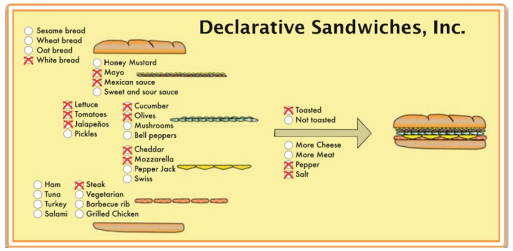
How would you approach this task?

15

15

One possible solution

1. Analyze the problem
   - Types of sandwiches in the store
   - Identify common and different characteristics of sandwiches
2. Design
   - Recipes for each type of sandwich
   - Ingredients and cooking procedures



© Apel, S., Batory, D., Kästner, C., & Saake, G. (2013). Feature-Oriented Software Product Lines: Concepts and Implementation (p. 308). Springer Berlin Heidelberg.

16

16

Introduction: Product Line Engineering



17

17

QUIZ: Can you distinguish the following definitions?

Match the following terms with their definitions:

Design ___

Engineering ___

Craft ___

Art ___

W. Skillful or artful contrivance applying measurable, reproductible, scientific and mathematical principles

X. Use of skill, taste and imagination in the production of aesthetic objects

Y. Deliberative, purposive planning

Z. Skilled occupation

18

18



19

At the beginning…



20

20

And then…



21

21

Reuse and production lines

Used in Venice in the 12th century

- to build ships with role specialization (each shipyard specializes in the production of certain parts, sells, buys what it needs and assembles)

Conceived in France during its revolution

- Fire arms production

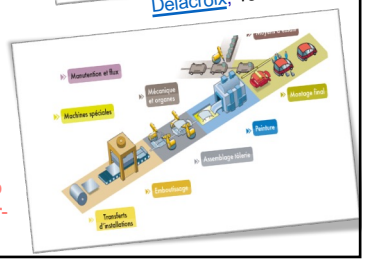
Pioneered in USA in late 19th century

Classical example

- Henry Ford Model T **production line**
- Produced from 1908 to 1927



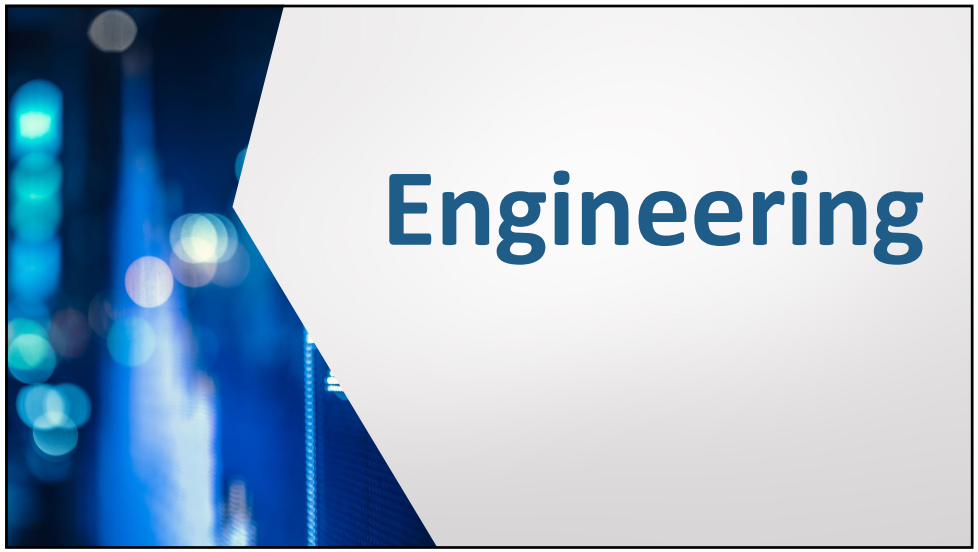
Delacroix, 1830



The problem with Production Lines is that they do not have the ability to deliver individualised (tailor-made as software industry use to do) products.

22

22



23



24



Garmisch conference (1968)

created the software engineering discipline and discussed many topics that are **challenges even now:**

1. **how to industrialize software production**
2. getting requirements right
3. designing adequate architectures
4. doing implementation effectively and correctly
5. verifying the quality of the result
6. maintaining software systems with targeted functionality and high code quality over

"Software production today appears in the scale of industrialization somewhere below the more backward construction industries. I think its proper place is considerably higher, and would like to investigate the prospects for ***mass-production techniques in software****."*

M. D. McIlroy. *Mass produced software components,* NATO-sponsored meeting on Software Engineering, Garmisch, Germany, 1968.

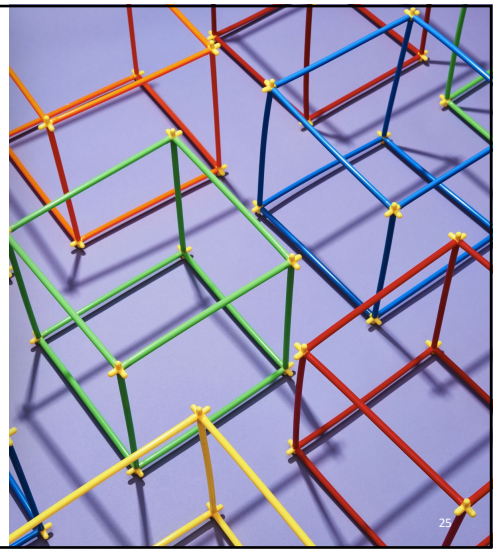
24

Software PL and families

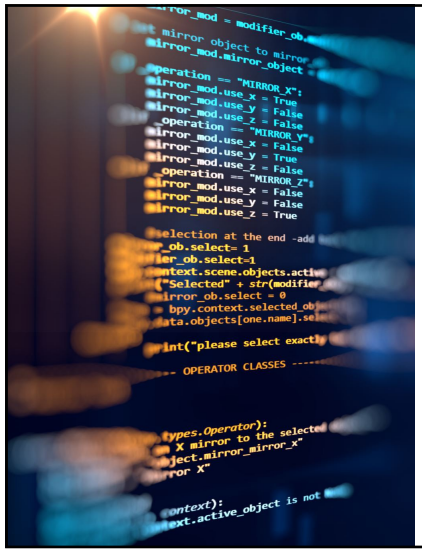
First reference to software product lines or families

*"**A program [therefore] should be conceived and understood as a member of a family**; it should be so structured out of components that various members of this family, sharing components, do not only share the correctness demonstration of the shared components but also of the shared substructure"*

Edsger W. Dijkstra, Notes in Structure Programming, 1969.



25



Product lines and software

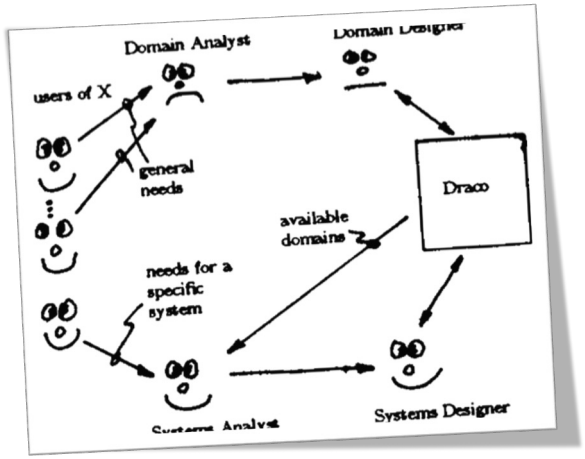
*"Program families are defined as set of programs whose common properties are so extensive that it is advantageous to study the **common properties** of the programs before analyzing the **individual members**."*

David Parnas. On the design and development of program families. IEEE transactions on software engineering. Vol. SE 2, N° 1. 1976.

26

26

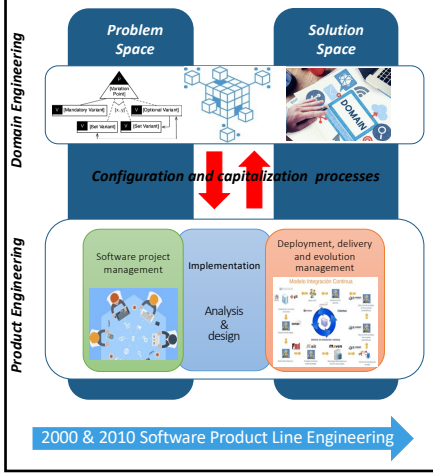
Neighbors (1989) introduced the concepts of domain and domain analysis, and provided the first systematic approach to domain engineering, known as Draco



27

27

Mass customization



Software is everywhere and for everything, so it must be mass-produced and customized. However, mass production does not meet this high demand of the market in terms of product customization.

Mass customization techniques, such as Product Line Engineering, are emerging, because **there is a demand for customized products to be manufactured at the price of mass-produced ones.**

(Mayer et al., 2016).

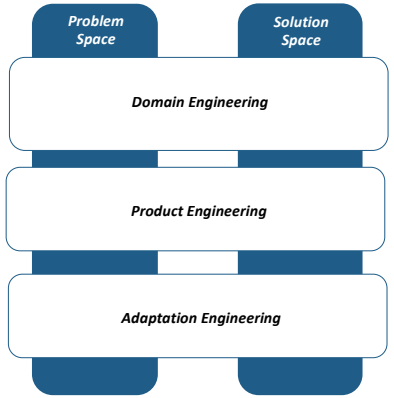
"A software product line is a set of **software-intensive** systems sharing a common, managed set of **features** that **satisfy the specific needs of a particular market segment** or mission and that are developed from a **common set of core assets** in a **prescribed way**."

SEI, 2001

28

29

Mass customization and *softwarisation*



"Nowadays, software is tightly coupled with physical processes and controls a lot of the technical and social activities of the world. So, a big challenge is **maintenance and resilience**. We have to understand how to design and implement reliable software such that maintenance, changeability, and variability is easier to handle. **A key is to work on software which is adaptive and to a large extent autonomous prepared for all the different use cases and their special demands that may come up".**

How will software engineering look tomorrow?
"Software engineering will be part of systems engineering of cyber physical systems closely connected to the real world".

2010 & 2020 … Domain-based Adaptive Cyber Physical Systems

Broy, Manfred. "Yesterday, today, and tomorrow: 50 years of software engineering." *IEEE Software* 35, no. 5 (2018): 38-43.

29



30

An historical perspective on reuse (1960-2000)

- 60s : subroutines
- 70s : modules
- 80s : objects
- 90s : components
- 2000 : services
- 2000 : product lines

Small grain focus
Results din't meet business goals

31

31

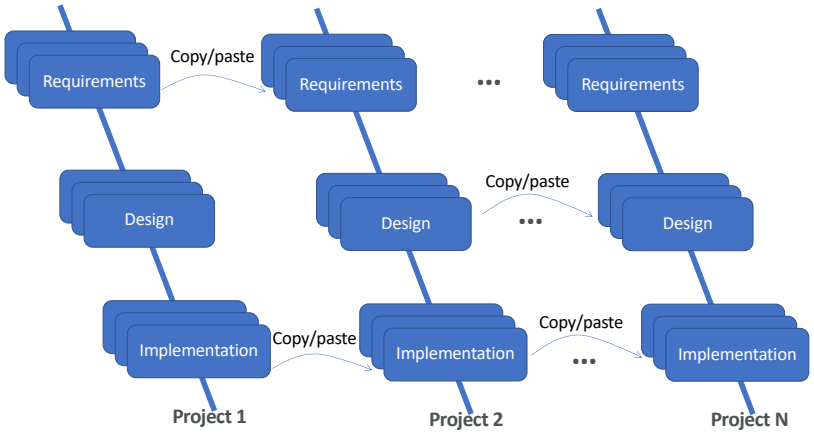
Six reuse mechanisms

1. Duplication (aka « copy & own »)
2. Δ based reuse
3. Evolution - Configuration management
4. Componenent-based reuse
5. Aspects
6. Product lines

32

32

Duplication-based Reuse



33

33

Pros/cons of duplication-based reuse

Pros.	Cons.
Simple. Without collateral effects on the other existing products.	**Hard to evolve**. This kind of reuse easily becomes a maintenance issue. «**Author's noise**» over each new product. **Oportunistic reuse** of artefacts (software, tests ..) : artefacts are not explicitly developed for reuse.

34

34

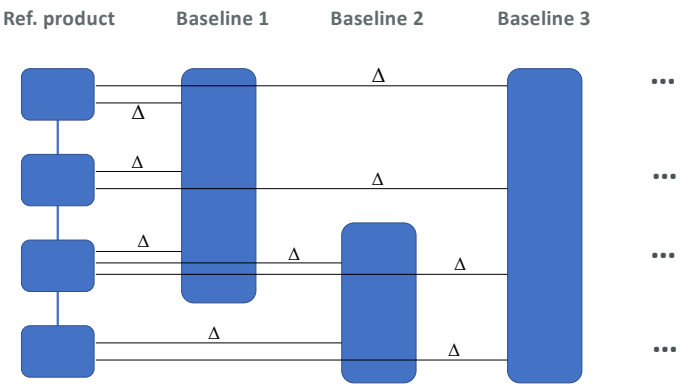
Duplication-based reuse VS PL

Duplication	PL.
Scatered view	Integrated view
Extensive specification	Intensive specification
Oportunistic reuse	Design for reuse
Multiple products that are viewed and maintained separately	The product line is treated as a whole
Product-level quality	Family-level quality

35

35

Δ based reuse

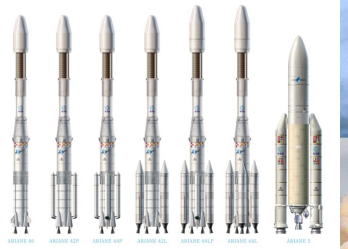


36

36

ROE : it doesn't always work well!

© Science Photo Library
Ariane 4 rocket versions





© Hackaday
Ariane 5

The Ariane rocket disaster of 1996, two reasons:

1. **A numeric overflow error**: attempt to fit a 64-bit format data (float) in 16-bit space (integer) and there was no exception handler associated with that conversion.
2. **Code reuse in a different context without testing**: as the facility that failed was not required for Ariane 5, there was no requirement associated with it and therefore there were no tests of that part of the software and hence no possibility of discovering the problem!

37

37

Pros/cons of Δ based reuse

Pros.	Cons.
Simple and sometimes very efficient (e.g., Mitsubishi Phones) Adapted for simple modifications over stables baselines Tooling is very efficient (e.g., HP QC, Polarion, etc.)	Non governance of common assets: how to manage a product "B" that wants to correct an error of an asset that inherits from a product "A"? Complexity explosion when the number of variants increases

38

38

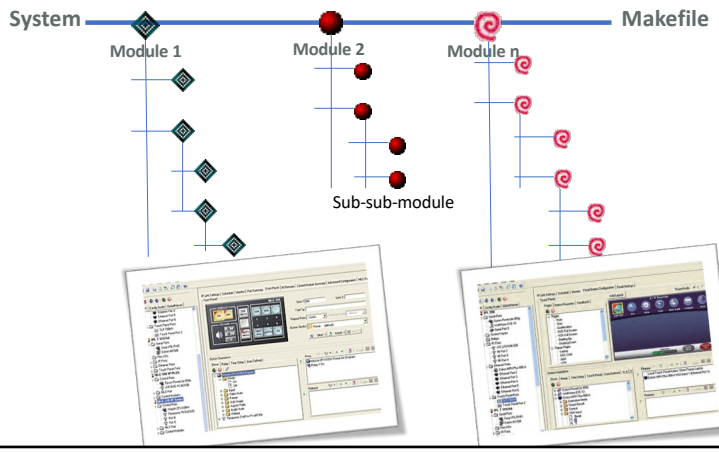
Δ based reuse VS PL

Δ based	PL.
One reference product evolution driven	Core architecture family driven
Δs are not predefined	Variability is specified in advance
Reuse is not necessarily planed	Reuse is strategically planned, and applies across the entire PL
No guaranty on quality	Some guaranties provided
No capitalization	Learning process
Vendors who release subsequent versions try to retire earlier versions as soon as they can	All versions are of interest

39

39

Configuration management



40

40

Configuration management

Pros.	Cons.
Low initial investment	Difficult compatibility management
Know baseline	Difficult to reuse at the level of transversal aspects
Individual work on concepts	Reuse on the context of one product

41

41

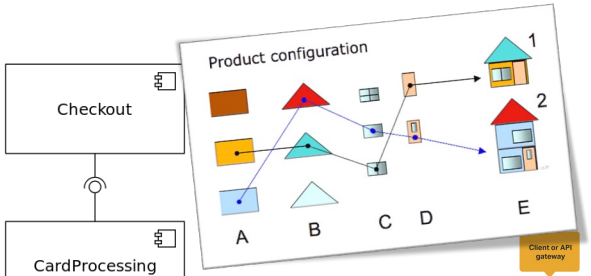
Configuration management VS PL

Configuration management	PL.
One reference product evolution driven	"PL is all about planning a plurality of products that will exist in the field and be maintained simultaneously" SEI, 2001
No guaranty on quality	Some guaranties provided
Reuse on the context of one product	Reuse on the context of one family of products
Configuration oriented	Prescription oriented

42

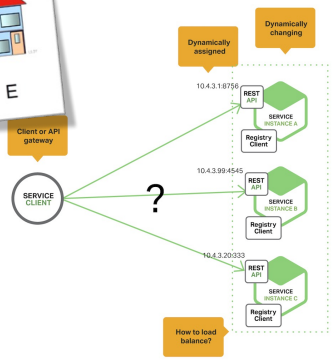
42

Component-based reuse



Two components expressed in UML 2.0.
The **checkout** component, responsible for facilitating the customer's order, ***requires*** the **card processing** component to charge the customer's credit/debit card (functionality that the latter *provides*).

Service discovery in a microservices architecture



43

43

Component-based reuse

Pros.	Cons.
Separation of concerns	No prescription on component choice
Black box view	No strategic view on assembling rules
Weak configuration	Knowledge of assembly is lost
Focus on interfaces	No guarantee on side effects
	No vision on cross cutting concerns

44

44

Component-based reuse VS PL

Component based	PL
Selection of components from a library or the marketplace to build products	Components in PL and the relationships among them are all specified by the product line architecture
Usually is missing strategic and systematic reuse	Product lines involve the strategic use of components

45

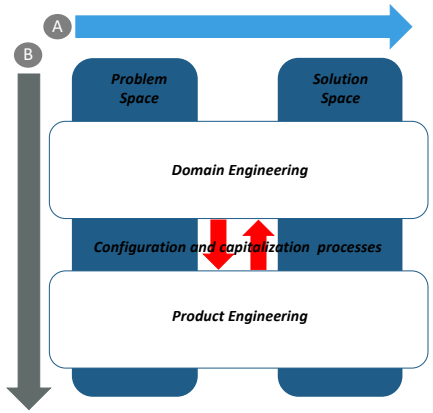
45

PLE reuse across problem and solution spaces

Software engineering is a problem-solving approach (A): *Problem definition* and *Problem solution*.

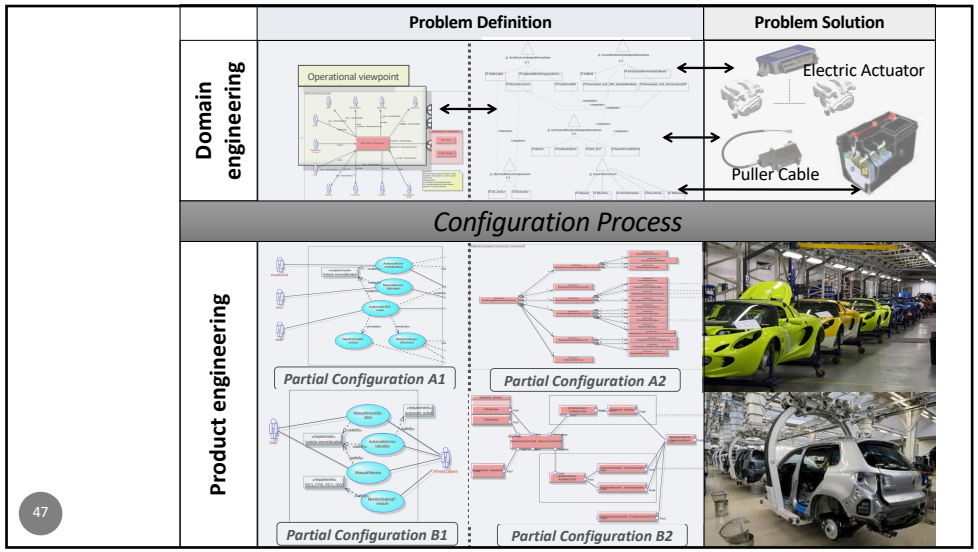
Product line engineering needs two different processes (B): *Domain engineering* and *Application Engineering*.

Configuration is a step-wise requirement process allowing deliver new *applications* that satisfy both: the domain constraints and the stakeholders' requirements.

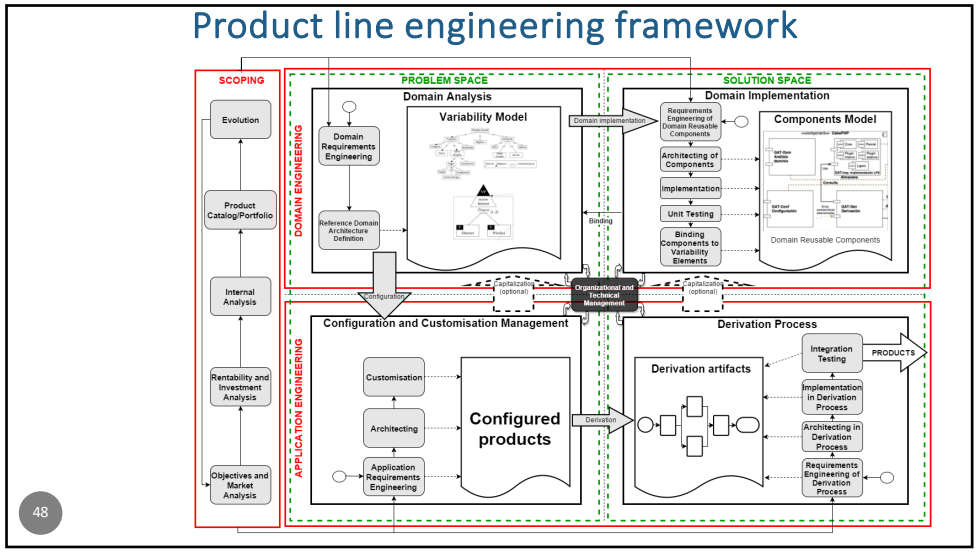


46

46

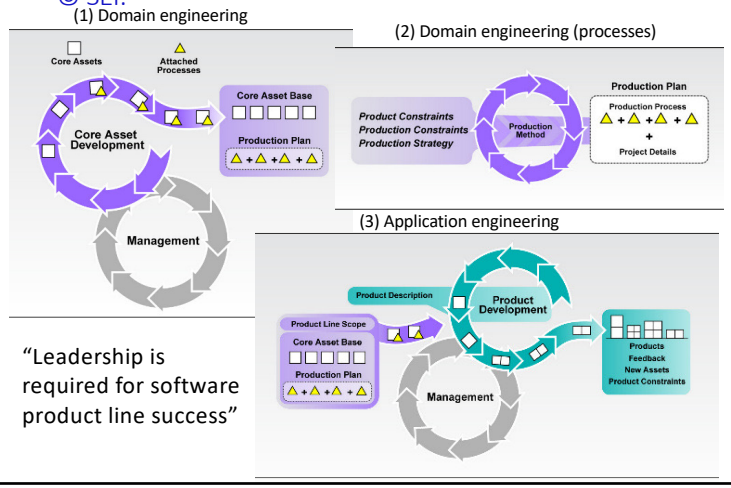

Problem Definition
Problem Solution
Domain engineering
Operational viewpoint
Electric Actuator
Puller Cable
Configuration Process
Product engineering
Partial Configuration A1
Partial Configuration A2
Partial Configuration B1
Partial Configuration B2
47

47


Product line engineering framework
SCOPING
Evolution
Product Catalog/Portfolio
Internal Analysis
Rentability and Investment Analysis
Objectives and Market Analysis
PROBLEM SPACE
SOLUTION SPACE
DOMAIN ENGINEERING
Domain Analysis
Variability Model
Domain Requirements Engineering
Reference Domain Architecture Definition
Domain Implementation
Requirements Engineering of Domain Reusable Components
Architecting of Components
Implementation
Unit Testing
Binding Components to Variability Elements
Components Model
Domain Reusable Components
Binding
Capitalization (optional)
Organizational and Technical Management
Configuration
APPLICATION ENGINEERING
Configuration and Customisation Management
Customisation
Architecting
Application Requirements Engineering
Configured products
Derivation
Derivation Process
Derivation artifacts
Integration Testing
PRODUCTS
Implementation in Derivation Process
Architecting in Derivation Process
Requirements Engineering of Derivation Process
48

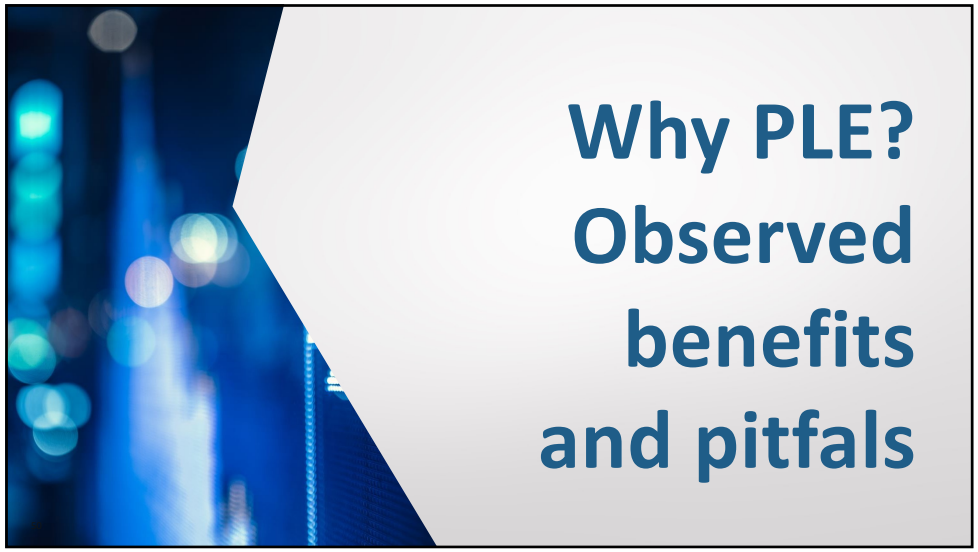
48

A Framework for Software Product Line Practice © SEI.



"Leadership is required for software product line success"

49



50

Product Lines Pros

- Improved **productivity** : 10x
- Increased **quality** : 10x
- Decreased **cost** : - 60%
- Decreased **labour needs** : - 87%
- Decreased **time to market** (to field, to launch) : - 98%
- Ability to **move into new markets** : months, not years

51

Source : SEI
https://insights.sei.cmu.edu/news/sei-software-product-lines-course-materials-now-freely-available/

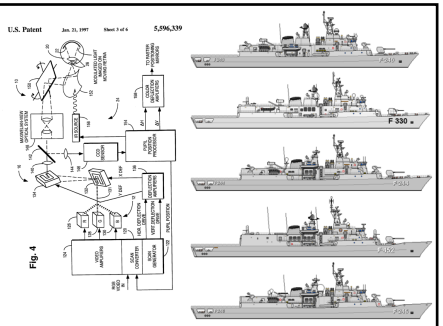
51

REX : CelsiusTech Systems

Hardware-to-software cost ratio changed from 35:65 to 80:20

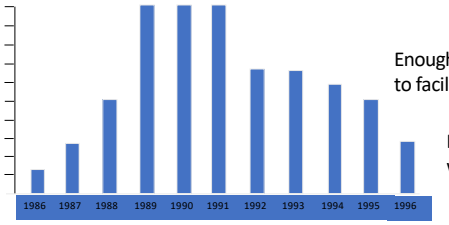
70 + % reuse

Over 55 variants built so far



The staff levels were too high: 150

Project staffing levels: 20 to 30 people



Enough numbers of reusable components to facilitate the design of other systems.

Designers were needed less and were reassigned to other projects.

52

52

Other Examples of Product Line Success

Motorola - FLEXworks Project (family of one-way pagers)
- 4x cycle time improvement
- 80% reuse

Hewlett Packard - printer systems
- reuse rate of about 70% for new products
- development time reduced by 67%
- -25% of the staff resources
- shipped 5x number of products
  - –that were 4x as complex
  - –and had 3x the number of features
  - –and 96% fewer defects

https://splc.net/fame.html



53

53

BAXTER Case study

Baxter International Inc. is a global healthcare company.

Treatments of complex medical conditions: **hemophilia, immune disorders**, kidney disease, cancer, trauma and other conditions

Conclusions from our work at Baxter (2009)

Domain engineer helps to retrieve BoM errors.

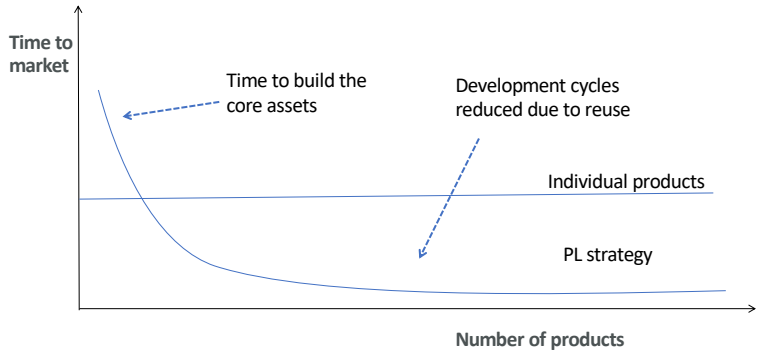
Financial Benefit at the ordering step estimated at: 50.000 Euros/ Year due to BOMs monthly check with this tool.



54

54

Decreased time to market (to field, to launch) by as much as 98%

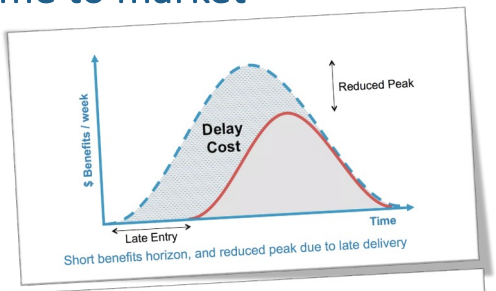


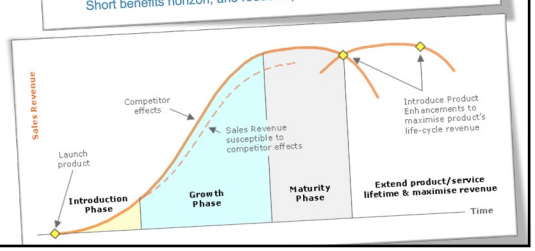
55

55

Minimizing the time to market

1. Greatly improved time to market.
2. Options to quickly develop new products.
3. The options and ability to growth in the market & capture new market niches.





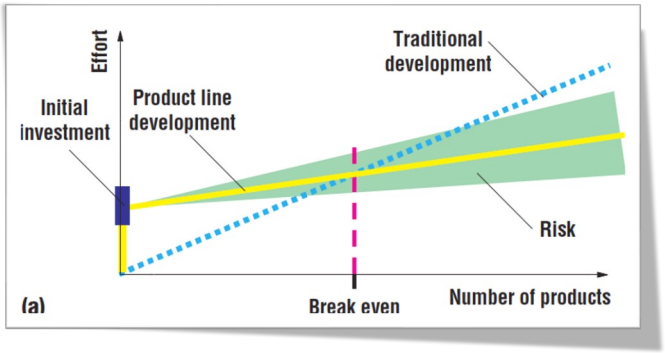
The revenue loss from being late to market can be much greater than the costs you incur designing and manufacturing your product!

56

56

Product line investment curves

(a)
big bang approach

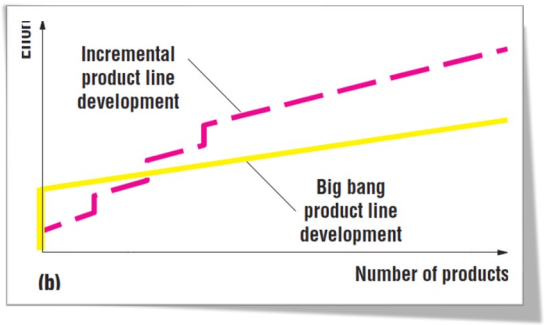


© Schmid K., Verlage M. The Economic Impact of Product Line Adoption and Evolution. IEEE SOFTWARE July/August 2002

57

57

Product line investment curves



(b)
big bang
VS
incremental approach

© Schmid K., Verlage M. The Economic Impact of Product Line Adoption and Evolution. IEEE SOFTWARE July/August 2002

58

58

Many stakeholders can see benefits from the PLE approach

Project management:

- Efficient use of workforce
- Ability to target new markets segments/technologies/products
- Maintain a cohesive structure throughout the product portfolio

59

59

Many stakeholders can see benefits from the PLE approach

Software development teams:

- Can focus on truly unique aspects of products
- Simplifies integration of software
- Fewer delays
- Avoidance of duplication of work

60

60

Costs and Benefits from the Requirements Point of View

	Benefit	Additional Costs
The requirements are written for the PL as a whole with: • Commonality • Variability	Requirements are reused and tailored New systems in the PL will be much simpler to specify Analysis and verification of requirements only once for all the systems.	Capturing requirements for a group of systems may require: • sophisticated analysis and • agree on common and variable requirements that are acceptable for all the systems.

61

61

Costs and Benefits from the Architecture Point of View

	Benefit	Additional Costs
The architecture defines how each product is assembled by means of components.	Design for a family of systems and not just for a single system. Reuse of the architecture for all the systems.	Variability of the PL imposes an additional constraint on the architecture and requires greater talent to define it.

62

62

Costs and Benefits from the Software Components Point of View

	Benefit	Additional Costs
The software components form the building blocks for each product in the PL Some will be reused without alteration. Others will be tailored.	The interfaces for components are reused. Capitalization of design decisions, data structures, algorithms, documentation, reviews, code, and debugging effort.	Components must be designed to be generic and extensible without loss of performance. Variation points must be anticipated.

63

63

Costs and Benefits from the Analysis, Verification, and Validation Point of View

	Benefit	Additional Costs
Analysis: extraction of information from the PL specification Verification: "Verification gets the requirements model right" [Boehm 81] Validation: "Validation gets the right requirements model" [Boehm 81]	Analysis, verification and validation have already been developed and reviewed for: • the components that are reused & • the architecture that relates these components	Analysis, verification and validation techniques appropriated for models with variability. Analysis, verification and validation may impose constraints on modifying, creating or synchronizing existing processes/models.

64

64

Costs and Benefits from the Tools and Processes Point of View

	Benefit	Additional Costs
The infrastructure for turning out a PL requires appropriate: **processes** and **tools.**	Reuse of software and process. Tools and environments purchased for one product can be amortized across the entire product line.	The boards, process, tools, and procedures must be more robust to manage product lines and single products.

65

65

Costs and Benefits from the skills and training Point of View

	Benefit	Additional Costs
Working on a single product ➔ working on the entire product line	Personnel can be more easily transferred among product projects as required. Productivity (number of products to which staff work applies) rises dramatically. Resources spent on training are expended only once.	Personnel must be trained beyond general system engineering to ensure that they understand software product line practices As product lines mature, the skills required tend to change (eg. from programming to relevant domain expertise and technology forecasting)

66

66

In Summary

1. Improved productivity by as much as 10x
2. Increased quality by as much as 10x
3. Decreased cost by as much as 60%
4. Decreased labour needs by as much as 87%
5. Decreased time to market (to field, to launch) by as much as 98%
6. Ability to move into new markets in months, not years

67

Recommended readings

Parnas, D.L., 1976. On the design and development of program families. *IEEE Transactions on software engineering*, (1), pp.1-9.

McGregor, J.D., Northrop, L.M., Jarrad, S. and Pohl, K., 2002. Initiating software product lines. *IEEE Software*, *19*(4), pp.24-27.

Birk, A., Heller, G., John, I., Schmid, K., von der Maßen, T. and Muller, K., 2003. Product line engineering, the state of the practice. *IEEE software*, *20*(6), pp.52-60.

Neighbors, J.M., 1984. The Draco approach to constructing software from reusable components. *IEEE Transactions on Software Engineering*, (5), pp.564-574.

Mazo Raúl. Guía para la adopción industrial de líneas de productos de software. *Editorial Eafit*, Medellín-Colombia, 2018

Schmid K., Verlage M. The economic impact of product line adoption and evolution. *IEEE software*, *19*(4), 2002.

Jeffrey Paulin. Measuring Software Reuse: Principles, Practices and Economic Models, Addison Wesley, 1997.

Rincon Luisa, Mazo Raúl, Salinesi Camille. Evaluating company's readiness for adopting product line engineering: a second evaluation round. *Complex Systems Informatics and Modeling Quarterly journal (CSIMQ)*, ISSN: 2255-9922, December, 2018.

Rincón Luisa, Mazo Raúl, Salinesi Camille. A multi-company empirical evaluation of a framework that evaluates the convenience of adopting product line engineering. Proceedings of the 24th ACM International Systems and Software Product Line Conference-Volume B. 2020

Rincón Luisa, Mazo Raúl, Salinesi Camille. Analyzing the convenience of adopting a product line engineering approach: an industrial qualitative evaluation. In Proceedings of the 23rd International Systems and Software Product Line Conference-Volume B, pp. 73-83, ACM, Paris-France, 2019.

68